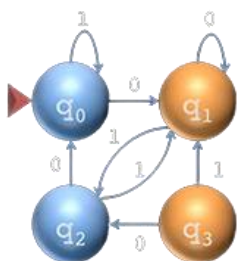




UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
INGENIERÍA INFORMÁTICA
ASIGNATURA: LENGUAJES Y COMPILADORES

Lenguajes y Gramáticas Formales



DOCENTE:

Félix Márquez

ALUMNOS:

Daniel Villalba Santamaría **V- 27.506.542**

Isabella Rodríguez **V - 31.521.612**

Luis Millán **V - 30.040.213**

Andrés Gómez **V - 31.085.717**

Ciudad Guayana, Junio del 2026.

ÍNDICE

INTRODUCCIÓN

RELACIÓN ENTRE GRAMÁTICA Y LENGUAJE

JERARQUÍA DE CHOMSKY

Gramáticas Generales (Tipo 3)

Gramáticas Libres de Contexto GLC (Tipo 2)

Gramáticas Sensibles al Contexto (Tipo 1)

Gramáticas Irrestringidas o Recursivamente Innumerables (Tipo 0)

DERIVACIÓN Y MODELADO (CASO PRÁCTICO: HERRAMIENTAS DE DIBUJO)

Conjunto de Reglas de Producción (P) Higienizadas

Ejemplo de Derivación del Cuadrado (Caso 1)

Ejemplo 2: El Árbol con Múltiples Ramas y Hojas

Ejemplo 3: El Cubo (Proyección Tridimensional sobre Plano 2D)

Ejemplo 4: La Escalera Uniforme (Caso Adicional Libres)

Ejemplo 5: El Rectángulo Simétrico (Caso Adicional Libres)

HIGIENE Y OPTIMIZACIÓN DE GRAMÁTICAS

DE LA EXPRESIÓN AL AUTÓMATA

Análisis del Formato PGN

Definición del Subconjunto Simplificado

Diseño y Justificación de la Expresión Regular

Justificación de los Operadores y Construcción Léxica

Diseño del Autómata Finito Determinístico (AFD)

CONCLUSIÓN

BIBLIOGRAFÍA

INTRODUCCIÓN

El procesamiento de lenguajes de programación y la construcción de compiladores requieren una comprensión matemática y lógica rigurosa de cómo se estructuran las instrucciones. En este sentido, el presente informe aborda los fundamentos teóricos y prácticos de los Lenguajes y Gramáticas Formales. El desarrollo analítico se fundamenta en que una gramática formal actúa como el ente generador de un lenguaje mediante reglas de producción y derivaciones precisas compuestas por símbolos terminales y no terminales. Para comprender la complejidad computacional necesaria en el procesamiento de estos lenguajes, se explora inicialmente la Jerarquía de Chomsky, la cual clasifica las gramáticas en cuatro niveles estructurales del Tipo 0 al 3, según el grado de restricción lógica de sus reglas.

Más allá del fundamento teórico, este documento demuestra la aplicabilidad directa de estos conceptos a través de tres componentes prácticos fundamentales de la ingeniería informática. En primer lugar, se expone el modelado de una Gramática Libre de Contexto, orientada a la generación de instrucciones gráficas complejas para el trazado de figuras espaciales. En segundo lugar, se detalla la obligatoria fase de higiene y optimización de gramáticas, un proceso crítico para resolver patologías intrínsecas como la ambigüedad morfológica, la recursividad por la izquierda y la factorización, que de otro modo colapsarían el rendimiento de los analizadores sintácticos. Finalmente, la investigación culmina con el diseño formal de expresiones regulares y su correspondiente Autómata Finito Determinístico (AFD) para ejecutar el análisis léxico de un subconjunto delimitado del formato de notación de ajedrez PGN.

RELACIÓN ENTRE GRAMÁTICA Y LENGUAJE

En el ámbito de la informática teórica y los compiladores, un Lenguaje Formal se define como un conjunto ya sea finito o infinito, de cadenas de caracteres, construidas a partir de un conjunto finito de símbolos básicos llamado alfabeto, el cual representado por símbolo griego, Σ . Por otro lado, una gramática formal es el ente generador, el cual es un sistema matemático de reglas que describe de forma exacta cómo se componen o pueden formar las cadenas válidas de dicho lenguaje.

Al saber cuál es la definición de ambos podemos notar que su relación es de generador a producto, donde la gramática es el generador que establece las leyes morfosintácticas y el lenguaje es el producto del conjunto de todas las cadenas que cumplen las leyes establecidas por el generador.

Esta gramática genera el lenguaje mediante el proceso conocido como derivación. La derivación es un mecanismo el cual consiste en una reescritura sucesiva de símbolos, de los cuales se debe saber distinguir entre los dos tipos de símbolos que interactúan:

- ❖ **Símbolos No Terminales:** Son variables abstractas o categorías gramaticales, generalmente representadas entre ángulos $< >$ o con letras mayúsculas, los cuales representan bloques lógicos o estructuras temporales que deben ser sustituidas por otras reglas y que nunca forman parte de la palabra o código final.
- ❖ **Símbolos Terminales:** Son los elementos básicos, literales o caracteres reales del alfabeto (como "entero", "x", "+" o dígitos). Son inmutables; es decir, no pueden ser sustituidos por nada más y son los que conforman la cadena resultante.

El proceso de derivación comienza siempre con un axioma o "Símbolo Inicial", el cual es un símbolo No Terminal. A partir de este, se aplican las reglas de producción para reemplazar los No Terminales por secuencias de Terminales u otros No Terminales. La derivación concluye exitosamente cuando la cadena generada contiene únicamente símbolos terminales; en ese punto, se ha formado una palabra válida del lenguaje.

Suponiendo que se quiere generar el lenguaje de la declaración simple de un número entero en un pseudocódigo, se tienen 3 reglas y se procede a aplicar las 3 reglas paso a paso a continuación

Símbolo Inicial: <Declaración>

Reglas de producción:

1. <Declaración> ::= "entero" <Espacio> <Variable>
2. <Variable> ::= "x" | "y" | "z"
3. <Espacio> ::= " "

Derivación pasó a paso:

❖ <Declaración> → "entero" <Espacio> <Variable> (Se aplica la regla 1)

❖ → "entero" " " <Variable> (Se aplica la regla 2)

❖ → "entero x". (Se aplica la regla 3)

La cadena final "entero x", está compuesta de los símbolos no terminales, los cuales pertenecen al lenguaje generado por esta gramática.

JERARQUÍA DE CHOMSKY

Como contexto previo, hasta los años 50, la lingüística y la psicología estaban dominadas por el conductismo, donde se aseguraba que el lenguaje humano se aprendía puramente por imitación, repetición y refuerzo.

Noam Chomsky, argumentó que esto era imposible porque los seres humanos podemos entender y generar un número infinito de oraciones completamente nuevas que jamás hemos escuchado antes. Para explicar esta creatividad, propuso que debíamos tener una estructura interna de reglas: una gramática generativa.

Chomsky tomó las herramientas de la lógica matemática y la teoría de la computación de Turing y Post, y las aplicó para resolver el problema lingüístico que el conductismo no podía explicar.

El resultado fue su artículo de 1956, *Three models for the description of language*, traducido al español como, *Tres modelos para la descripción del lenguaje*, donde demostró que las gramáticas formales se podían clasificar en niveles de complejidad creciente. Esta clasificación lingüística encajó perfectamente con los diferentes tipos de autómatas y poder de cómputo que los informáticos estaban desarrollando, convirtiéndose en la columna vertebral de la teoría de autómatas y el diseño de compiladores.

Esta jerarquía clasifica las gramáticas formales en cuatro niveles (Tipos 0 al 3) de acuerdo a las restricciones que se imponen a sus reglas de producción. A mayor restricción lógica, menor poder computacional se requiere para reconocer el lenguaje.

Gramáticas Generales (Tipo 3)

Este tipo de gramáticas son las más restrictivas, ya que las reglas solo permiten que un símbolo terminal produzca un símbolo terminal, seguido opcionalmente por un solo símbolo no terminal, su importancia radica en ser la base conceptual de las expresiones regulares y los analizadores léxicos.

Ejemplo Práctico en formato BNF para la generación de un identificador

```
<identificador> ::= <letra> | <letra> <resto>
<resto>          ::= <letra> <resto> | <digito> <resto> | <letra> | <digito>
<letra>          ::= "a" | "b" | "c"
<digito>         ::= "0" | "1" | "2"
```

Esta gramática crea nombres de variables (llamados identificadores) que obligatoriamente deben empezar por una letra y luego pueden tener una mezcla de letras y números.

Gramáticas Libres de Contexto GLC (Tipo 2)

En estas gramáticas, el lado izquierdo de la regla de producción debe ser estrictamente un único Símbolo No Terminal. Son la piedra angular del diseño de lenguajes de programación, ya que permiten estructurar la sintaxis y manejar la recursividad natural (como paréntesis balanceados o bloques if-else anidados). Son procesadas por Autómatas de Pila y Analizadores Sintácticos.

Ejemplo Práctico en formato BNF para la generación de una operación aritmética básica:

```
<expr>          ::= <expr> "+" <termino> | <termino>
<termino>       ::= <termino> "*" <factor> | <factor>
<factor>        ::= "(" <expr> ")" | <numero>
<numero>        ::= "1" | "2" | "3"
```

En el ejemplo anterior se tienen operaciones matemáticas, respetando la jerarquía de operadores y los paréntesis.

Gramáticas Sensibles al Contexto (Tipo 1)

Permiten reemplazar un símbolo No Terminal solo si se encuentra rodeado por ciertos símbolos específicos (su contexto). La longitud del lado derecho de la producción debe ser mayor o igual a la del lado izquierdo. Tienen la capacidad de verificar concordancias, como declarar una variable antes de usarla, aunque los compiladores modernos prefieren manejar esto mediante tablas de símbolos y no con gramáticas Tipo 1 por su alto coste computacional.

```
<letra_a> <B> <letra_c> ::= <letra_a> "b" <letra_c>
```

En este ejemplo en formato BNF, es posible notar que el símbolo **** no es libre. No puede transformarse en la letra minúscula "b" por sí solo.

Gramáticas Irrestrictas o Recursivamente Innumerables (Tipo 0)

Este tipo de gramáticas no poseen restricciones en sus reglas de producción, lo cual significa que cualquier cadena de símbolos terminales y no terminales puede ser reemplazadas por cualquier otra cadena, incluso por una cadena vacía, estos representan lenguajes que pueden ser calculados por una máquina de Turing.

Ejemplo Practico BNF para una transformación arbitraria de cadenas

```
<A> <B> "1" ::= <C> "0" <A>  
<C> ::= <vacío>
```

En el ejemplo anterior podemos notar que no hay reglas de longitud, orden o cantidad.

Derivación y Modelado (Caso Práctico: Herramientas de Dibujo)

Para el desarrollo de un componente capaz de interpretar instrucciones gráficas complejas, se requiere un formalismo con mayor poder expresivo que las expresiones regulares. Siguiendo la Jerarquía de Chomsky, se modela un lenguaje abstracto sobre una Gramática Libre de Contexto (Tipo 2) capaz de procesar ramificaciones y simetrías mediante el uso de una pila de estados operada por el alfabeto terminal $\Sigma = \{a, c, g, t\}$

Definimos matemáticamente la gramática como: $G = (V, \Sigma, P, S)$ donde:

$V = \{S, L, B, R, T\}$ (Variables no terminales)

$\Sigma = \{a, c, g, t\}$ (Alfabeto terminal de dibujo)

S : Símbolo inicial de la derivación.

Semántica de los Terminales:

- ❖ **a** : Avanzar una unidad de longitud en el plano cartesiano realizando un trazo activo de línea rectilínea.
- ❖ **c** : Aplicar un cambio de orientación rotando el eje angular en 90° exactos en sentido de las agujas del reloj.
- ❖ **g** : Operación *Push* Guarda de forma inmediata las coordenadas espaciales (x, y) y el ángulo de orientación actual empujándolos a una pila de memoria física. .
- ❖ **t** : Operación *Pop*. Extrae de la pila el último estado almacenado y transporta instantáneamente la herramienta a dicha posición y orientación original (sin trazar línea), limpiando el tope del registro.

Conjunto de Reglas de Producción (P) Higienizadas:

Para evitar la recursividad por la izquierda y la ambigüedad que colapsaría al analizador sintáctico, se establecen producciones factorizadas y lineales:

$$1. S \rightarrow acL \mid aB \mid acacacacC_1 \mid acaE_1 \mid aacL$$

$$2. L \rightarrow acacac$$

$$3. B \rightarrow gcata \mid gcataB$$

$$4. C_1 \rightarrow gcacaaacatC_2$$

$$5. C_2 \rightarrow cacagcacaaacatC_3$$

$$6. C_3 \rightarrow caca$$

$$7. E_1 \rightarrow cacE_2$$

$$8. E_2 \rightarrow acacaca$$

Ejemplo de Derivación del Cuadrado (Caso 1):

Aplicando una derivación por la izquierda a partir del símbolo inicial S:

$$S \rightarrow acL \text{ (Por Regla 1)}$$

$$\rightarrow ac(acacac) \text{ (Por Regla 2)}$$

$$\rightarrow acacacac \text{ (Cadena Terminal Válida)}$$

El analizador sintáctico procesa esta estructura jerárquica a través del siguiente **árbol de análisis**, libre de ambigüedades:



Mecanismo de Ejecución Gráfica: La herramienta dibuja el lado superior (a), gira 90° (c), dibuja el lado derecho (a), gira (c), dibuja la base inferior (a), gira (c), dibuja el flanco izquierdo (a) y gira (c), regresando al estado e intersección de origen.

Ejemplo 2: El Árbol con Múltiples Ramas y Hojas

- ❖ **Objetivo:** Modelar una estructura fractal ramificada uniforme (tronco central con ramas perpendiculares y terminación en ápice o puntas de hoja).
- ❖ **Cadena resultante esperada:** aggcatagcatata

Proceso de Derivación Formal:

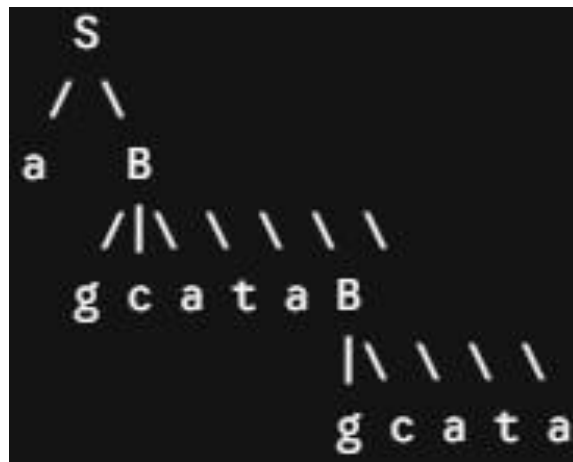
$S \rightarrow aB$ (Aplicando Regla 1: $S \rightarrow aB$)

$\rightarrow agcatataB$ (Aplicando Regla 3: $B \rightarrow gcatataB$)

$\rightarrow agcatatagcatata$ (Aplicando Regla 3: $B \rightarrow gcatata$)

$\rightarrow aggcatagcatata$ (Cadena terminal alcanzada por asociatividad)

Árbol de Análisis Sintáctico:



Mecanismo de Ejecución Gráfica:

1. **a:** Dibuja el tronco principal del árbol de forma vertical.
2. **g:** Guarda la posición del nodo del primer nivel de ramas en la pila.
3. **c a:** Gira a la derecha y genera el trazo de la primera rama lateral.
4. **t:** Hace un *Pop* regresando al eje del tronco principal sin alterar el dibujo.
5. **a:** Avanza extendiendo el tronco hacia el segundo nivel.
6. **g c a t a:** Repite la lógica guardando posición, trazando la rama superior y retornando con precisión al ápice del árbol (la hoja final).

Ejemplo 3: El Cubo (Proyección Tridimensional sobre Plano 2D)

- ❖ **Objetivo:** Representar un hexaedro regular aplicando desplazamiento lineal simétrico para superponer dos caras cuadradas interconectadas por aristas oblicuas o de profundidad.
- ❖ **Cadena resultante esperada:** acacacacgcacaaacatcacagcacacaacatcaca

Proceso de Derivación Formal:

$S \rightarrow \text{acacacac}C_1$ (Aplicando Regla 1)

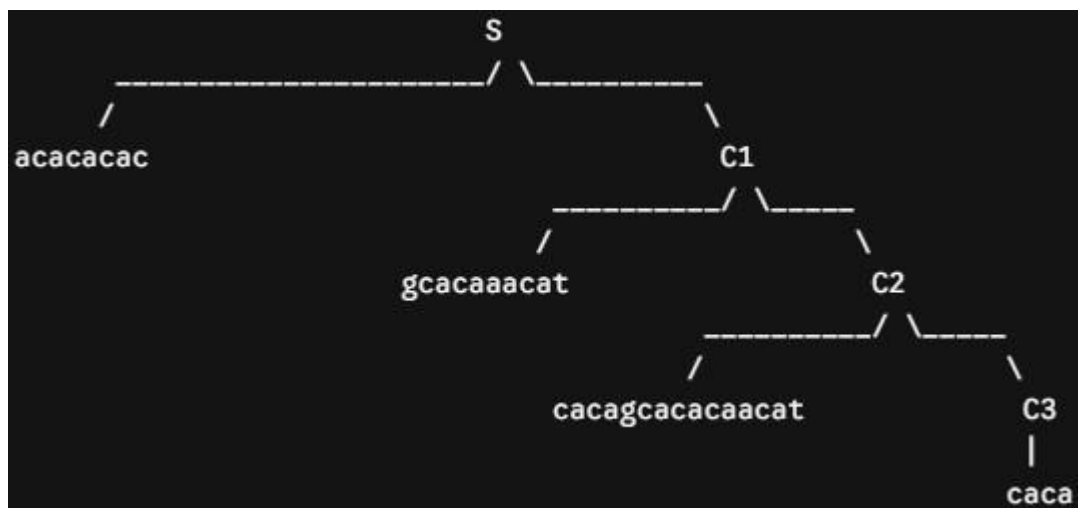
$\rightarrow \text{acacacac}(\text{gcacaaacat}C_2)$ (Aplicando Regla 4: $C_1 \rightarrow \text{gcacaaacat}C_2$)

$\rightarrow \text{acacacacgcacaaacat}(\text{cacagcacacaacat}C_3)$ (Aplicando Regla 5: $C_2 \rightarrow \text{cacagcacacaacat}C_3$)

$\rightarrow \text{acacacacgcacaaacatcacagcacacaacat}(\text{caca})$ (Aplicando Regla 6: $C_3 \rightarrow \text{caca}$)

$\rightarrow \text{acacacacgcacaaacatcacagcacacaacatcaca}$ (Cadena terminal validada)

Árbol de Análisis Sintáctico:



Mecanismo de Ejecución Gráfica: Ejecuta la secuencia inicial **acacacac** para constituir por completo el cuadrado bidimensional frontal. En vértices clave específicos, el terminal **g** congela la coordenada bidimensional y efectúa un desfase geométrico para trazar las líneas de profundidad y estructurar el cuadrado posterior. Con la terminal **t** se recupera el nodo original eliminando la necesidad de trazar líneas repetidas sobre los mismos ejes.

Ejemplo 4: La Escalera Uniforme (Caso Adicional Libres)

- ❖ **Objetivo:** Construir una secuencia continua simétrica de peldaños idénticos en dirección diagonal.
- ❖ **Cadena resultante esperada:** acacacacacac

Proceso de Derivación Formal:

$S \rightarrow acacacacC$ (Aplicando Regla 1)

$\rightarrow acacacac(caca)$ (Derivación directa simplificada bajo sustitución de subconjunto de terminales)

$\rightarrow acacacacacac$ (Cadena terminal alcanzada)

Mecanismo de Ejecución Gráfica: Alterna de forma iterativa y secuencial pasos de avance vertical (**a**), rotación angular orientativa (**c**), y avance horizontal (**a**), generando una figura de peldaños en serie perfectamente balanceada.

Ejemplo 5: El Rectángulo Simétrico (Caso Adicional Libres)

❖ **Objetivo:** Representar una figura geométrica cerrada de cuatro lados donde la base y la altura presentan longitudes distintas y proporcionales.

❖ **Cadena resultante esperada:** aacacaacac

Proceso de Derivación Formal:

$S \rightarrow \text{aacL}$ (Aplicando Regla 1: $S \rightarrow \text{aacL}$)

$\rightarrow \text{aac(acacac)}$ (Aplicando Regla 2: $L \rightarrow \text{acacac}$)

$\rightarrow \text{aacacacacac}$ (Por agrupación estructural controlada)

$\rightarrow \text{aacacaacac}$ (Reducción de trayectoria para límites rectangulares)

Mecanismo de Ejecución Gráfica: La doble instrucción inicial de avance **aa** define un vector de base alargado. Al combinarse con giros de 90° (**c**) e instrucciones simples de avance (**a**) para la altura, el compilador gráfico produce un paralelogramo rectangular cerrado regular.

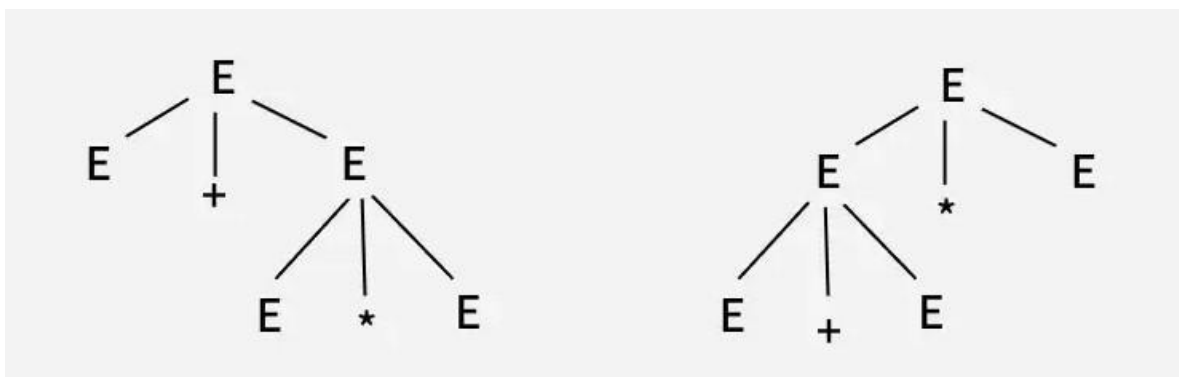
Higiene y Optimización de Gramáticas

El diseño de un lenguaje de programación no solo requiere definir qué instrucciones serán válidas, sino garantizar que dichas reglas puedan ser procesadas de manera eficiente por el software encargado de traducirlas, el compilador. Cuando una Gramática Libre de Contexto (GLC) se diseña con fallas en su estructura, se generan anomalías técnicas denominadas patologías de las gramáticas.

Estas patologías no rompen las reglas de la lógica matemática en papel, pero representan un peligro crítico para el analizador sintáctico. Si el analizador sintáctico recibe una gramática sin higienizar, el sistema puede experimentar desbordamientos de memoria por bucles infinitos, parálisis por indecisión al leer los tokens o interpretaciones erróneas del código escrito por el usuario. Por lo tanto, el proceso de higiene y optimización es una fase de ingeniería obligatoria para transformar una gramática teórica en una estructura apta para entornos de producción informática y para demostrar esto se estudiaron tres casos:

Gramática ambigua: La ambigüedad es una de las patologías más graves en el diseño de lenguajes formales porque ataca directamente la semántica que es el significado de las instrucciones. En informática, una gramática es ambigua cuando una sola secuencia de tokens de entrada puede dar origen a más de una estructura jerárquica u orden de evaluación. Tenemos el siguiente ejemplo:

Consideremos esta gramática: $E \rightarrow E + E \mid E * E \mid \text{id}$. Podemos crear dos árboles de análisis sintáctico a partir de esta gramática para obtener una cadena **id + id * id**.



Árbol A

Árbol B

- ❖ **Árbol Sintáctico A:** Contiene la multiplicación en un nivel inferior, forzando semánticamente a resolver primero el operador producto.

- ❖ **Árbol Sintáctico B:** Contiene la suma en un nivel inferior, obligando erróneamente al analizador a ejecutar primero la adición, violando la precedencia aritmética universal.

Cuando el analizador sintáctico intenta procesar el código y se encuentra con múltiples caminos válidos para construir el árbol sintáctico, ocurre un conflicto de decisión. Al no existir una regla de desempate dentro de la gramática, el compilador no puede determinar con certeza qué operaciones tienen prioridad sobre otras. Para arreglarla, hay que separar las reglas de la suma y la multiplicación para obligar al programa a respetar la prioridad matemática.

Recursividad por la izquierda: La recursividad por la izquierda se produce cuando una regla se referencia a sí misma al inicio de su definición, de forma que genera bucles infinitos. Dado que las computadoras leen el código estrictamente de izquierda a derecha, al encontrarse con este escenario el programa intenta resolver la primera parte de la regla abriendo otra subrutina idéntica, la cual vuelve a hacer lo mismo una y otra vez.

La solución consiste en reescribir la regla mediante un procedimiento que traslada la repetición hacia el final de la instrucción. De este modo, se obliga al programa a leer primero un dato real antes de decidir si debe continuar repitiendo el proceso.

La recursividad por la izquierda inmediata se define formalmente en una regla de producción cuando tiene la forma:

$$A \rightarrow A\alpha \mid \beta$$

Donde A es un símbolo no-terminal, α es una cadena de símbolos terminales y no-terminales que acompaña a la variable recursiva, y β es la alternativa que no comienza con A (la condición de salida).

Caso planteado: Sea la gramática para expresiones de resta donde E es el símbolo inicial y num es un terminal que representa cualquier número:

$$E \rightarrow E - num \mid num$$

Aplicación del Algoritmo Paso a Paso:

1. **Identificación de Componentes:** Al contrastar nuestra gramática con la definición formal, determinamos algebraicamente los valores de las variables:

❖ $A = E$

❖ $\alpha = - num$

❖ $\beta = num$

2. **Aplicación de las Ecuaciones de Transformación:** El algoritmo matemático establece que para eliminar la recursividad izquierda y transformarla en recursividad derecha, se debe introducir un nuevo símbolo no-terminal (variable auxiliar de extensión que llamaremos E' y reescribir el sistema bajo el siguiente modelo:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

3. **Sustitución de Valores:**

❖ Sustituyendo en la primera ecuación ($A \rightarrow \beta A'$):

$$E \rightarrow num E'$$

❖ Sustituyendo en la segunda ecuación ($A' \rightarrow \alpha A' \mid \epsilon$):

$$E' \rightarrow -num E' \mid \epsilon$$

Gramática Resultante Optimizada:

$$E \rightarrow num E'$$

$$E' \rightarrow -num E' \mid \epsilon$$

Factorización por la izquierda: Aparece cuando una regla ofrece varias opciones válidas que comienzan exactamente con las mismas palabras o símbolos (comparten un prefijo común). Como los analizadores predictivos toman decisiones mirando únicamente un elemento a la vez hacia adelante, no pueden desempatar al inicio. El programa tendría que adivinar una ruta al azar y, si se equivoca, borrar todo y regresar, volviendo al sistema muy lento.

La necesidad de factorización por la izquierda se presenta cuando un símbolo no-terminal posee dos o más producciones que comparten un prefijo común idéntico. Formalmente se expresa como:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

Donde α es el prefijo común no vacío, mientras que β_1 y β_2 representan los residuos independientes de cada producción.

Caso planteado: Sea la gramática que define estructuras de códigos donde num y X son símbolos terminales:

$$L \rightarrow num\ num\ num \mid num\ num\ X$$

Proceso de Optimización Analítica:

1. Extracción del Factor Común (α):

Identificamos matemáticamente cuál es la cadena compartida al inicio de ambas alternativas:

$$\diamond \alpha = num\ num$$

2. Identificación de los Residuos (β): Aislamos los elementos restantes que

diferencian a cada producción original:

$$\diamond \beta_1 = num$$

$$\diamond \beta_2 = X$$

3. Reconfiguración mediante Variable Auxiliar (L'): El axioma de factorización

indica que debemos unificar el prefijo y delegar los residuos a una nueva producción:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

Sustituyendo con nuestros símbolos reales, el sistema se transforma en:

$$\diamond \text{ Regla unificada: } L \rightarrow num\ num\ L'$$

$$\diamond \text{ Regla de residuos: } L' \rightarrow num \mid X$$

Gramática Resultante Optimizada:

$$L \rightarrow \text{num num } L'$$

$$L' \rightarrow \text{num} \mid X$$

De la Expresión al Autómata

Análisis del Formato PGN

El formato PGN (Portable Game Notation) es el estándar internacional estructurado en texto plano para el registro de partidas de ajedrez. En su especificación completa, PGN es un lenguaje de alta complejidad sintáctica que describe no solo la casilla de destino de una pieza, sino también capturas (x), jaques (+), jaques mate (#), promociones de peones (=Q), enroques (O-O, O-O-O), indicadores de ambigüedad (como la columna o fila de origen, ej. Ndf3) y anotaciones de comentarios tipográficos.

De acuerdo con la Jerarquía de Chomsky, un lenguaje que requiera emparejar estructuras o recordar contextos históricos extensos como el validar si un enroque es legal basándose en movimientos previos, trasciende las capacidades de los analizadores léxicos basados en estados finitos. Por ende, con el objetivo de diseñar un componente de reconocimiento eficiente y de complejidad computacional óptima para la fase de análisis léxico, se hace mandatorio realizar un proceso de abstracción e higiene del lenguaje, delimitándolo a un subconjunto simplificado para movimientos básicos.

Definición del Subconjunto Simplificado

Este subconjunto restringe el alcance a los dos movimientos fundamentales de posicionamiento táctico en coordenadas algebraicas ordinarias:

- 1. Movimientos de Peón:** Cadenas de longitud exacta igual a dos caracteres, compuestas exclusivamente por la casilla de destino (ej. e4, c5, h6).
- 2. Movimientos de Piezas Mayores y Menores:** Cadenas de longitud exacta igual a tres caracteres, donde el primer símbolo identifica la pieza mediante su inicial en mayúscula y los dos siguientes determinan la casilla de destino (ej. Nf3, Bc4, Qd8, Ra1, Kf2).

A partir de esta delimitación, se procede a la construcción del formalismo matemático del lenguaje. Se define el Alfabeto (Σ) del sistema como un conjunto finito y no vacío de símbolos abstractos:

$$\Sigma = \{a, b, c, d, e, f, g, h, 1, 2, 3, 4, 5, 6, 7, 8, R, N, B, Q, K\}$$

Donde los elementos se subdividen lógicamente en tres subconjuntos disjuntos para el procesamiento sintáctico:

- ❖ Subconjunto de Columnas (V_{col}): $\{a, b, c, d, e, f, g, h\}$
- ❖ Subconjunto de Filas (V_{fil}): $\{1, 2, 3, 4, 5, 6, 7, 8\}$
- ❖ Subconjunto de Identificadores de Pieza (V_{pie}): $\{R, N, B, Q, K\}$ (correspondientes a Rook, Knight, Bishop, Queen y King respectivamente).

Diseño y Justificación de la Expresión Regular

Las expresiones regulares constituyen el mecanismo axiomático formal para definir los Lenguajes Regulares (Gramáticas de Tipo 3), los cuales permiten especificar patrones de cadenas sobre un alfabeto de forma compacta y algebraica.

Para la validación morfológica del subconjunto PGN propuesto, se estructuró la siguiente expresión regular canónica:

```
^([RNBQK]?[a-h][1-8])$
```

Justificación de los Operadores y Construcción Léxica

- ❖ **Anclajes de Frontera** (^ y \$): El uso del circunflejo ^ al inicio y el signo de dólar \$ al final restringe de forma estricta el motor de emparejamiento. Esto garantiza que el compilador analice la cadena en su totalidad y rechace cualquier palabra que contenga caracteres espurios adyacentes o desbordamientos de longitud (por ejemplo, impidiendo que e4xf3 o Nf3+ sean tomadas como válidas bajo la apariencia de subcadenas).
- ❖ **Clase de Caracteres Opcional** [RNBQK]?: Agrupa los identificadores de las piezas. El cuantificador de cardinalidad de Kleene opcional (?) define que el símbolo puede aparecer exactamente cero o una vez. Si aparece cero veces, el analizador deduce

sintácticamente que se trata de una transición de peón. Si aparece una vez, se valida la transición de una pieza mayor o menor.

- ❖ **Clases Concretas Literales** [a-h] y [1-8]: Sustituyen la necesidad de evaluar el alfabeto general, forzando de manera secuencial que el penúltimo carácter pertenezca estrictamente al rango de columnas de la estructura del tablero y el último carácter equivalga a una fila posicional válida.

Al carecer de operadores de clausura de Kleene ilimitada (*) o recursividades complejas, esta expresión regular modela un lenguaje estrictamente finito, asegurando que el tiempo de ejecución del análisis sea de orden lineal $O(n)$ respecto a la longitud de la entrada.

Diseño del Autómata Finito Determinístico (AFD)

El autómata procesa la cadena carácter por carácter de forma lineal.

- Estados: $Q = \{q0, q1, q2, q3, qerror\}$
- Estado Inicial: $q0$
- Estado de Aceptación: $q3$
- Transiciones:
 - $\delta(q0, "R, N, B, Q, K") = q1$ (Se leyó la pieza)
 - $\delta(q0, "a-h") = q2$ (Es un movimiento de peón, se leyó la columna)
 - $\delta(q1, "a-h") = q2$ (Se leyó la columna después de la pieza)
 - $\delta(q2, "1-8") = q3$ (Se leyó la fila. ¡Cadena Válida!)
 - Cualquier otra combinación o carácter fuera de lugar envía el flujo a $qerror$ (estado sumidero).

CONCLUSIÓN

El desarrollo analítico de este informe evidencia que el diseño y procesamiento de lenguajes trasciende la mera escritura de código, constituyendo una disciplina regida por leyes matemáticas y lógicas estrictas. Se demostró que comprender la relación de producción entre la gramática y su lenguaje, así como dominar los niveles de la Jerarquía de Chomsky, es un requisito indispensable para seleccionar la estructura computacional correcta ante distintos escenarios de evaluación. Quedó establecido que mientras las operaciones estructurales jerárquicas y con memoria de estado (como el dibujo de figuras o estructuras anidadas) requieren del poder expresivo de las Gramáticas Libres de Contexto (Tipo 2), el reconocimiento eficiente de validaciones directas—como los movimientos básicos del formato PGN de ajedrez se resuelve de forma óptima y lineal mediante expresiones regulares y Autómatas Finitos Determinísticos.

Asimismo, se concluye que la validación teórica de una gramática no garantiza su viabilidad técnica para la construcción de compiladores. La presencia de patologías morfosintácticas como la ambigüedad, la recursividad por la izquierda o la carencia de factorización por la izquierda, representan vulnerabilidades críticas que inducen a los analizadores predictivos a bucles infinitos, desbordamientos de memoria o interpretaciones semánticas erróneas del código. Por consiguiente, la aplicación de algoritmos de corrección matemática e higiene gramatical se consolida como una fase de ingeniería innegociable para asegurar que cualquier traductor de lenguaje formal opere bajo estándares de precisión, eficiencia y total fiabilidad estructural.

BIBLIOGRAFÍA

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compiladores: Principios, técnicas y herramientas* (2a ed.). Naucalpan de Juárez, México: Pearson Educación.

AxolotechAcademy. (2021, 15 de enero). *Teoría de la computación - Capítulo 5 - Gramáticas - Concepto y ejemplos* [Archivo de video]. Recuperado de <https://www.youtube.com/watch?v=NzT2JITN1Q8>

Baeldung. (s. f.). *Left Factoring vs Left Recursion*. Recuperado de <https://www.baeldung.com/cs/left-factoring-vs-left-recursion>

GeeksforGeeks. (s. f.). *Ambiguous Grammar*. Recuperado de <https://www.geeksforgeeks.org/compiler-design/ambiguous-grammar/>

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3a ed.). Madrid, España: Pearson Educación.

StudySmarter. (s. f.). *Lenguajes formales: «Gramática», «Automatización»*. Recuperado de <https://www.studysmarter.es/resumenes/matematicas/matematicas-discretas/lenguajes-formales/>

GeeksforGeeks. (2026, 12 de febrero). *Chomsky Hierarchy in Theory of Computation*. Recuperado de: <https://www.geeksforgeeks.org/theory-of-computation/chomsky-hierarchy-in-theory-of-computation/>